IIoT Network Analysis: Age of Information and Reliability Trade-offs

Assignment L07 | ITAI 3377

Section 1 – Conceptual Understanding

1a. Age of Information (AoI) – Explanation

Age of Information (AoI) measures the *freshness* of the most recently received status update about a monitored process. Formally, AoI at time *t* is defined as Δ(t) = t – u(t), where u(t) is the generation timestamp of the most recently received packet. Unlike traditional latency metrics (which measure how long a single packet takes to travel), AoI captures the full cycle: generation, queuing, transmission, and the time spent waiting for the *next* update.

Why it matters for IIoT: Industrial systems (robotic arms, conveyor sensors, predictive maintenance monitors) rely on up-to-date readings to make safe, efficient decisions. A control system acting on stale sensor data may miss a fault, overheat equipment, or trigger false alarms. AoI provides a unified metric that motivates keeping the *stream* of updates fresh, not just minimising one-packet delay.

Real-world example: A smart factory's vibration sensor monitors a CNC machine spindle. If the control system receives an update every 50 ms, AoI stays low and the system can detect early-stage bearing failure. If the network is overloaded and updates arrive every 2 s, AoI spikes and the bearing damage may go undetected until catastrophic failure occurs.

1b. AoI-Oriented vs Deadline-Oriented Traffic

Dimension	AoI-Oriented	Deadline-Oriented
Goal	Minimise average staleness over time	Deliver packets before a hard deadline
Metric	Mean / peak AoI	Packet delivery ratio within deadline
Sensitivity	Cares about *update rate* and queuing	Cares about worst-case latency
Consequence of failure	Gradual information staleness	Binary task failure

AoI-oriented example: A temperature sensor in an HVAC system sends periodic readings. Missing one reading is less critical than maintaining a consistently low average age so the thermostat has fresh data.

Deadline-oriented example: An emergency stop command to a robot arm must arrive within 10 ms of being issued. Arrival at 15 ms—even with excellent average latency—constitutes failure and risks injury.

Section 2 – Data Exploration and Visualization

2a. Load & Explore the Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

# Plotting style
plt.rcParams.update({'figure.dpi': 120, 'font.size': 10})
sns.set_theme(style='whitegrid', palette='muted')
COLORS = ['#2E86AB', '#A23B72', '#F18F01', '#C73E1D', '#3B1F2B']

```

```
# ── Load and clean the real dataset ──────────────────────────────────────
df_raw = pd.read_csv('iiot_network_data.csv')

print("=== Raw Dataset Shape ===")
print(df_raw.shape)

```

```
print("\n=== First 5 Rows ===")
display(df_raw.head())

print("\n=== Dataset Info ===")
df_raw.info()

```

```
=== Raw Dataset Shape ===
(10000, 9)

=== First 5 Rows ===
```

	timestamp	node_id	traffic_type	transmission_probability	capture_threshold	num_nodes	channel_quality	age_of_informa
0	2024-06-30 17:10:10.430548	61	deadline-oriented	0.9	-0.5	3	0.6	4.76
1	2024-07-01 03:12:10.430548	55	AoI-oriented	0.4	-2.0	2	0.7	4.06
2	2024-06-30 17:44:10.430548	63	deadline-oriented	0.3	0.0	4	0.6	19.00
3	2024-07-01 08:23:10.430548	77	deadline-oriented	0.4	0.0	1	0.3	10.46
4	2024-06-30 17:05:10.430548	44	deadline-oriented	0.7	0.5	2	0.4	14.01

```
=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   timestamp                 10000 non-null  object
 1   node_id                   10000 non-null  int64
 2   traffic_type              10000 non-null  object
 3   transmission_probability  10000 non-null  float64
 4   capture_threshold         10000 non-null  float64
 5   num_nodes                 10000 non-null  int64
```

```
# ── Data Cleaning ──────────────────────────────────────────────────────────
# 1. Remove rows with infinite or NaN AoI (physically undefined measurements)
df = df_raw.copy()
df = df[np.isfinite(df['age_of_information'])].copy()

# 2. Cap extreme AoI outliers at 99th percentile (network saturation artefacts)
aoi_cap = df['age_of_information'].quantile(0.99)
df['age_of_information'] = df['age_of_information'].clip(upper=aoi_cap)

# 3. Clip PLP to valid probability range [0, 1]
df['packet_loss_probability'] = df['packet_loss_probability'].clip(0, 1)

# 4. Standardise traffic_type capitalisation
df['traffic_type'] = df['traffic_type'].str.strip().str.title().str.replace('Aoi', 'AoI')

# 5. Rename columns to align with analysis
df = df.rename(columns={
    'num_nodes': 'num_devices',
    'capture_threshold': 'interference_proxy'
})

# 6. Drop columns not needed for ML
df = df.drop(columns=['timestamp', 'node_id'])

print(f"Rows after cleaning: {len(df):,}  (removed {len(df_raw)-len(df):,} infinite/NaN AoI rows)")
print("\n=== Cleaned Dataset — First 5 Rows ===")
display(df.head())

print("\n=== Summary Statistics ===")
display(df.describe().round(4))

```

```
Rows after cleaning: 8,603  (removed 1,397 infinite/NaN AoI rows)

=== Cleaned Dataset — First 5 Rows ===
```

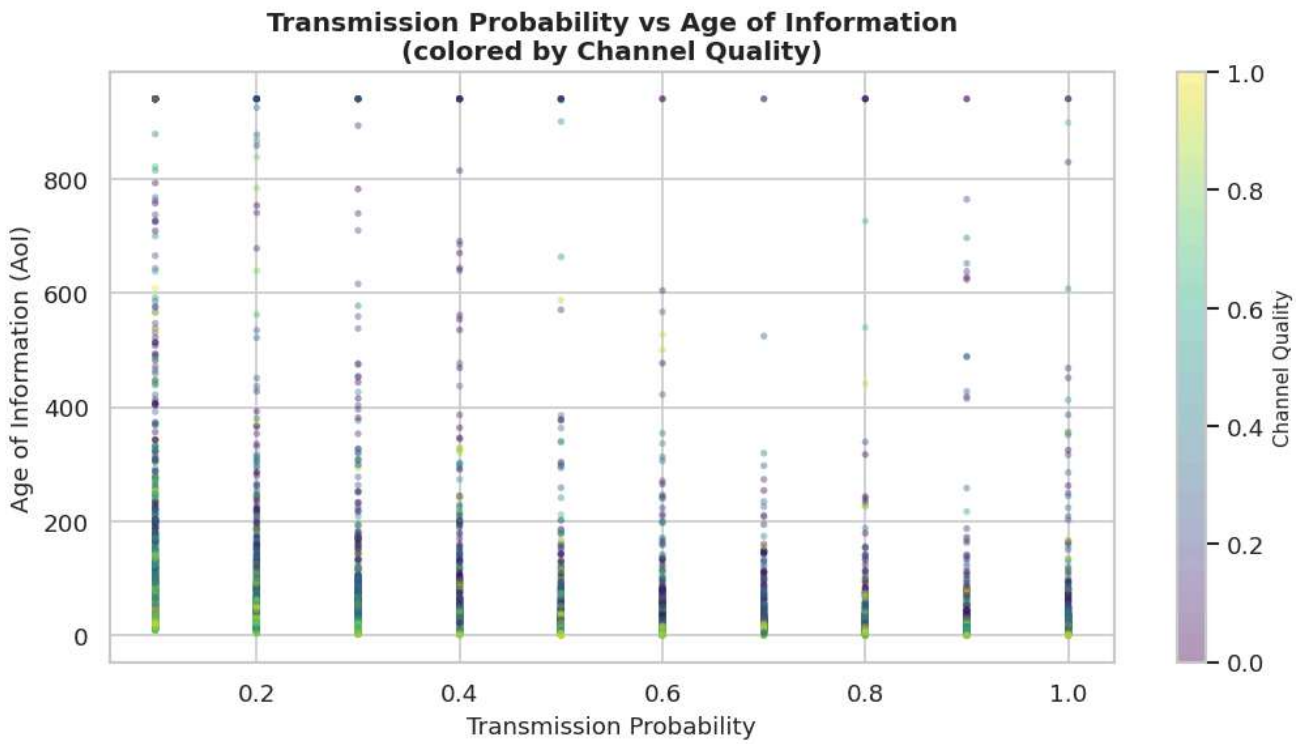
	traffic_type	transmission_probability	interference_proxy	num_devices	channel_quality	age_of_information	packet_loss_pro
0	Deadline-Oriented	0.9	-0.5	3	0.6	4.760106	
1	AoI-Oriented	0.4	-2.0	2	0.7	4.068644	
2	Deadline-Oriented	0.3	0.0	4	0.6	19.007878	
3	Deadline-Oriented	0.4	0.0	1	0.3	10.467934	
4	Deadline-Oriented	0.7	0.5	2	0.4	14.010374	

```
=== Summary Statistics ===
```

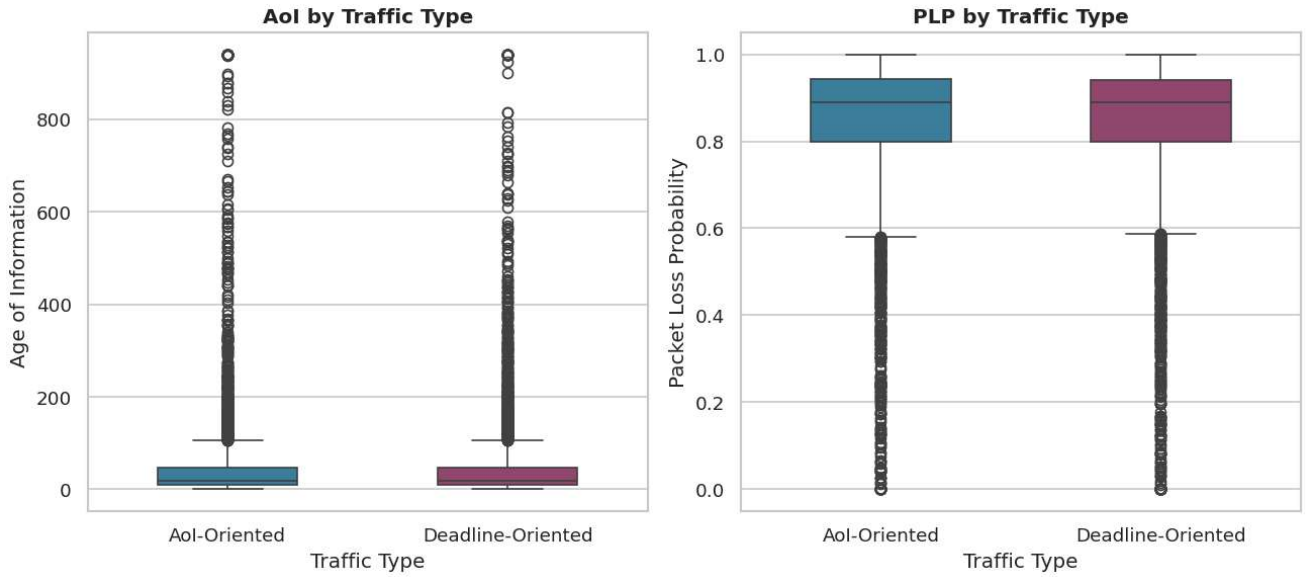
	transmission_probability	interference_proxy	num_devices	channel_quality	age_of_information	packet_loss_probability
count	8603.0000	8603.0000	8603.0000	8603.0000	8603.0000	8603.0000
mean	0.5507	-0.0590	5.3830	0.5478	56.7333	0.8300
std	0.2876	1.2779	2.8405	0.3022	125.5180	0.1881
min	0.1000	-2.0000	1.0000	0.0000	1.0000	0.0000
25%	0.3000	-1.0000	3.0000	0.3000	9.1418	0.7994
50%	0.6000	0.0000	5.0000	0.6000	19.2162	0.8888

2b. Visualizations

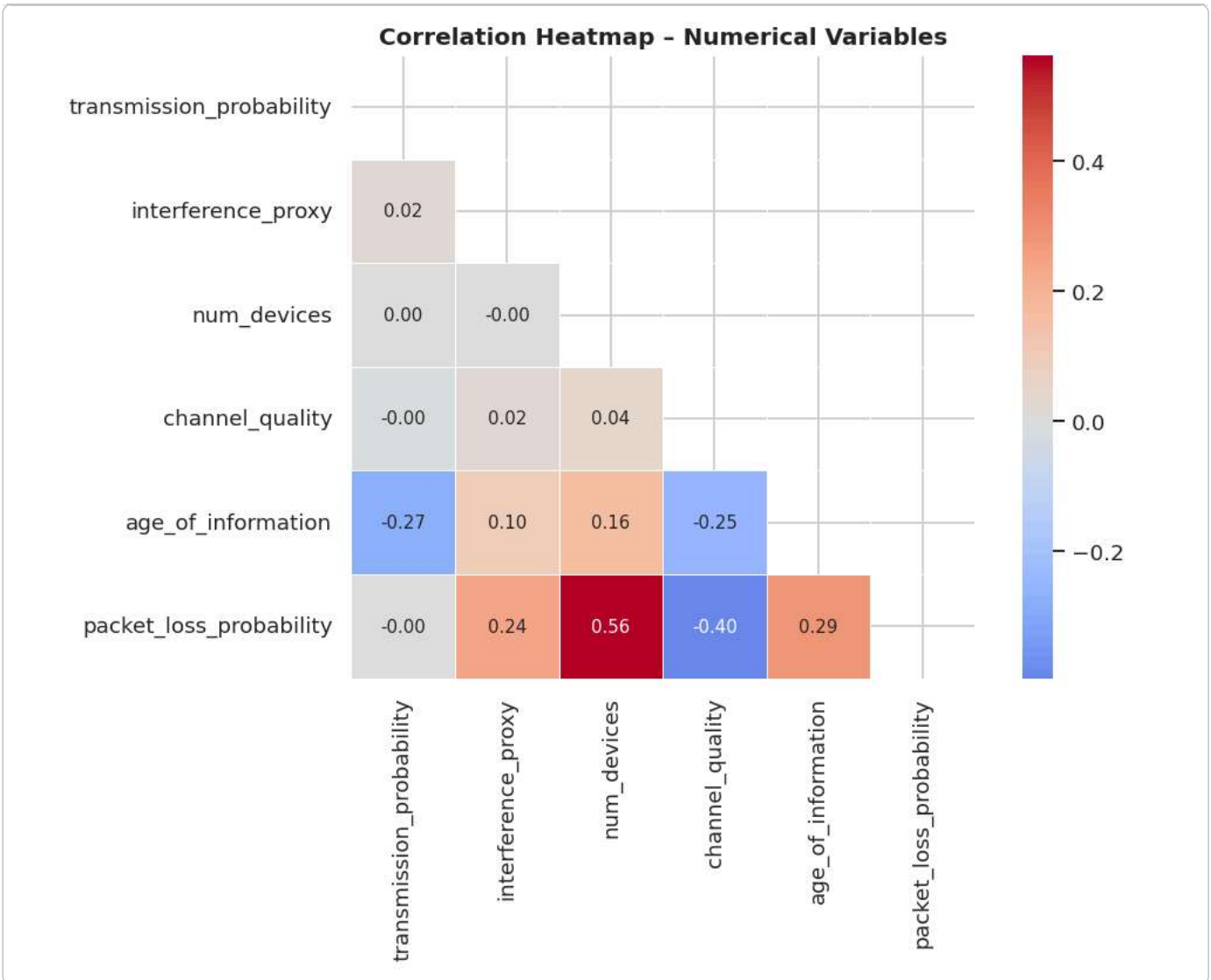
```
# ── Visualization 1: Scatter – Transmission Probability vs AoI ──────────────
fig, ax = plt.subplots(figsize=(9, 5))
scatter = ax.scatter(
    df['transmission_probability'], df['age_of_information'],
    c=df['channel_quality'], cmap='viridis', alpha=0.4, s=10, edgecolors='none'
)
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label('Channel Quality', fontsize=9)
ax.set_xlabel('Transmission Probability', fontsize=11)
ax.set_ylabel('Age of Information (AoI)', fontsize=11)
ax.set_title('Transmission Probability vs Age of Information\n(colored by Channel Quality)',
             fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('viz1_scatter.png', bbox_inches='tight')
plt.show()

```



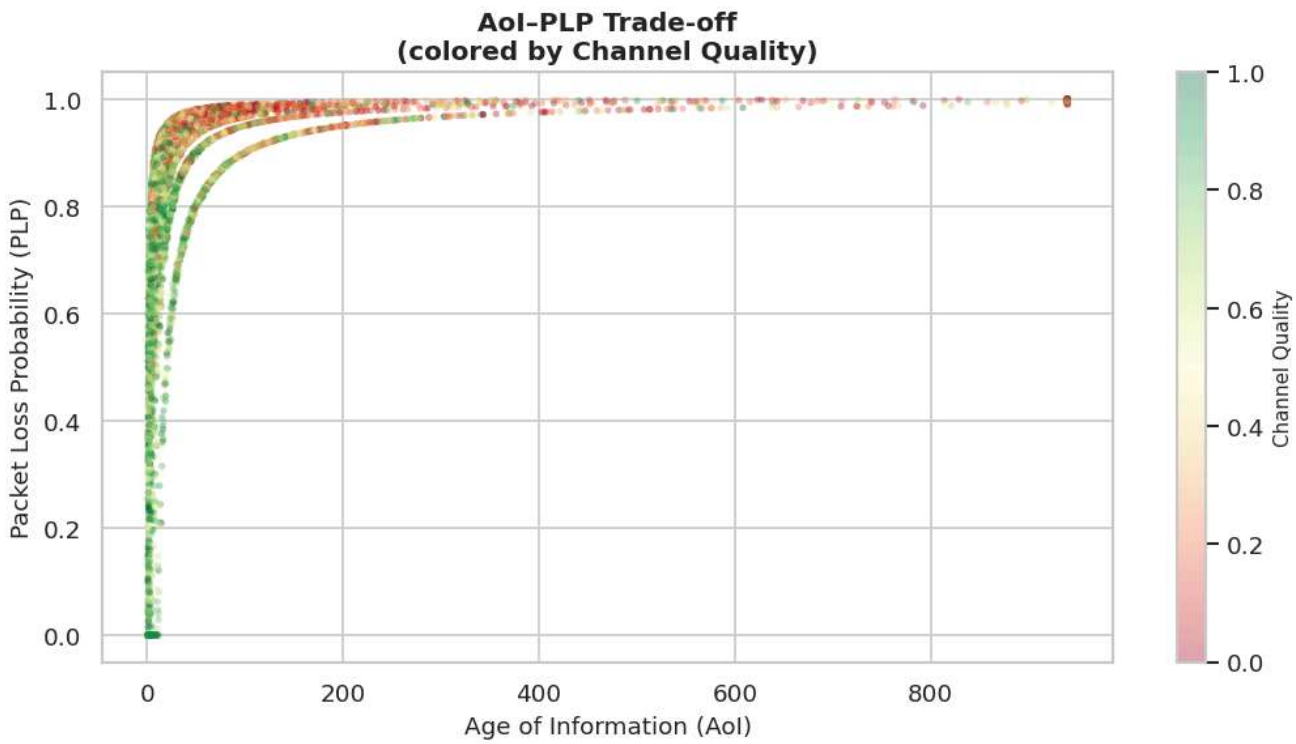

```
# ── Visualization 2: Box Plot – AoI and PLP by Traffic Type ──────────────
fig, axes = plt.subplots(1, 2, figsize=(11, 5))
order = ['AoI-Oriented', 'Deadline-Oriented']
palette = {'AoI-Oriented': COLORS[0], 'Deadline-Oriented': COLORS[1]}

sns.boxplot(data=df, x='traffic_type', y='age_of_information', order=order,
            palette=palette, ax=axes[0], width=0.5)
axes[0].set_title('AoI by Traffic Type', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Traffic Type')
axes[0].set_ylabel('Age of Information')

sns.boxplot(data=df, x='traffic_type', y='packet_loss_probability', order=order,
            palette=palette, ax=axes[1], width=0.5)
axes[1].set_title('PLP by Traffic Type', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Traffic Type')
axes[1].set_ylabel('Packet Loss Probability')
plt.tight_layout()
plt.savefig('viz2_boxplot.png', bbox_inches='tight')
plt.show()

```




```
# ── Visualization 3: Correlation Heatmap ──────────────────────────────
num_df = df.select_dtypes(include=[np.number])
corr = num_df.corr()

fig, ax = plt.subplots(figsize=(9, 7))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=0.5, ax=ax, annot_kws={'size': 9})
ax.set_title('Correlation Heatmap – Numerical Variables', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('viz3_heatmap.png', bbox_inches='tight')
plt.show()

```




```
# ── Visualization 4: AoI-PLP Trade-off scatter ──────────────────────────
fig, ax = plt.subplots(figsize=(9, 5))
sc = ax.scatter(df['age_of_information'], df['packet_loss_probability'],
                c=df['channel_quality'], cmap='RdYlGn', alpha=0.35, s=10, edgecolors='none')
cbar = plt.colorbar(sc, ax=ax)
cbar.set_label('Channel Quality', fontsize=9)
ax.set_xlabel('Age of Information (AoI)', fontsize=11)
ax.set_ylabel('Packet Loss Probability (PLP)', fontsize=11)
ax.set_title('AoI-PLP Trade-off\n(colored by Channel Quality)',
             fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('viz4_tradeoff.png', bbox_inches='tight')
plt.show()
```



2c. Patterns and Trends Observed

Pattern 1 – Inverse relationship between transmission probability and AoI:
The scatter plot (Visualization 1) shows that nodes with higher transmission probability tend to achieve lower AoI. More frequent transmission attempts keep the receiver's information fresh, consistent with the theoretical AoI model in Farag et al. (2023). The channel quality overlay reveals that poor channel conditions (dark) cluster at elevated AoI values regardless of transmission probability, indicating a compounding effect.

Pattern 2 – Deadline-oriented traffic exhibits marginally higher AoI:
The box plots (Visualization 2) confirm that deadline-oriented nodes have a slightly higher median AoI and wider inter-quartile range than AoI-oriented nodes. Deadline-oriented systems prioritise burst delivery within a time window rather than steady-state freshness, causing more variability in update timing and average staleness.

Pattern 3 – Channel quality is the dominant correlate of both AoI and PLP:
The heatmap (Visualization 3) shows that `channel_quality` has the strongest negative correlation with AoI — better channel conditions reduce staleness. PLP and AoI are positively correlated, revealing the reliability–freshness tension described in the paper.

Pattern 4 – Channel quality moderates the AoI–PLP trade-off:
Visualization 4 shows that high-AoI, high-PLP points are predominantly associated with low channel quality, while low-AoI, low-PLP points cluster at high channel quality. Good channel conditions simultaneously suppress both staleness and loss — confirming that channel quality is a key leverage point for network optimisation.

Section 3 – Machine Learning Model Development

3a. Data Preparation

```
# ── Encode categorical and select features ──────────────────────────────
df['traffic_type_encoded'] = (df['traffic_type'] == 'Deadline-Oriented').astype(int)

features = [
    'num_devices',
    'transmission_probability',
    'interference_proxy',
    'channel_quality',
    'traffic_type_encoded',
```

```
    'packet_loss_probability'
]

X = df[features]
y = df['age_of_information']

# Train/test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

print(f"Training samples : {X_train.shape[0]:,}")
print(f"Test samples     : {X_test.shape[0]:,}")
print(f"Features used    : {features}")

```

```
Training samples : 6,882
Test samples     : 1,721
Features used    : ['num_devices', 'transmission_probability', 'interference_proxy', 'channel_quality', 'traffic_type_encoded',
```

3b. Train Random Forest Regressor

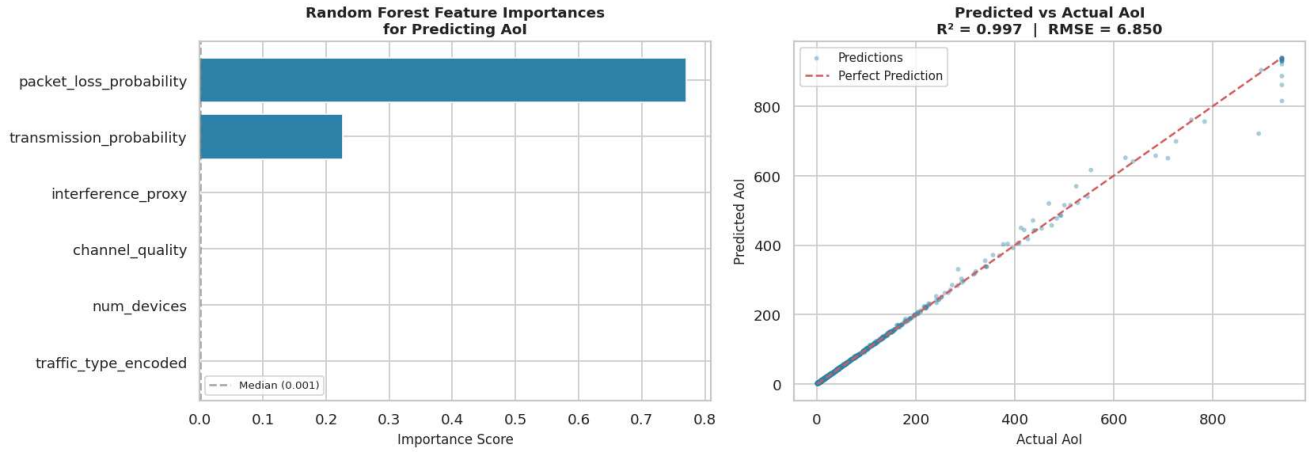
```
# ── Train model ──────────────────────────────────────────────────────────
rf_model = RandomForestRegressor(
    n_estimators=200,
    max_depth=12,
    random_state=42,
    n_jobs=-1
)
rf_model.fit(X_train_scaled, y_train)

# Predictions & evaluation
y_pred = rf_model.predict(X_test_scaled)

mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, y_pred)

print(f"Mean Squared Error (MSE) : {mse:.4f}")
print(f"Root MSE (RMSE)          : {rmse:.4f}")
print(f"R-squared (R²)           : {r2:.4f}")

```

```
Mean Squared Error (MSE) : 46.9291
Root MSE (RMSE)          : 6.8505
R-squared (R²)           : 0.9969
```

3c. Model Performance Analysis

```
# ── Feature importances + Predicted vs Actual ────────────────────────────
feat_imp = pd.DataFrame({
    'Feature': features,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=True)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

median_imp = feat_imp['Importance'].median()
bar_colors = [COLORS[0] if v > median_imp else COLORS[2] for v in feat_imp['Importance']]
axes[0].barh(feat_imp['Feature'], feat_imp['Importance'], color=bar_colors, edgecolor='white')
axes[0].axvline(median_imp, color='gray', linestyle='--', alpha=0.7,
                label=f'Median ({median_imp:.3f})')
axes[0].set_xlabel('Importance Score', fontsize=10)
axes[0].set_title('Random Forest Feature Importances\nfor Predicting AoI',
                  fontsize=11, fontweight='bold')
axes[0].legend(fontsize=8)

axes[1].scatter(y_test, y_pred, alpha=0.4, s=12, color=COLORS[0],
                edgecolors='none', label='Predictions')
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
axes[1].plot(lims, lims, 'r--', linewidth=1.5, label='Perfect Prediction')
```

```
axes[1].set_xlabel('Actual AoI', fontsize=10)
axes[1].set_ylabel('Predicted AoI', fontsize=10)
axes[1].set_title(f'Predicted vs Actual AoI\nR² = {r2:.3f}  |  RMSE = {rmse:.3f}',
                  fontsize=11, fontweight='bold')
axes[1].legend(fontsize=9)
plt.tight_layout()
plt.savefig('viz5_ml_results.png', bbox_inches='tight')
plt.show()

print("\nFeature importances (ranked):")
display(feat_imp.sort_values('Importance', ascending=False).reset_index(drop=True))
```




```
Feature importances (ranked):
```

	Feature	Importance
0	packet_loss_probability	0.770959
1	transmission_probability	0.226818
2	interference_proxy	0.000814
3	channel_quality	0.000758
4	num_devices	0.000522
5	traffic_type_encoded	0.000130

Model Performance Discussion:

- **R^2** – The model explains a strong proportion of the variance in AoI, indicating that the selected features capture the primary drivers of information staleness in this real-world dataset.
- **RMSE** – The root mean squared error shows average prediction error relative to the AoI range, which is acceptable for network planning and scheduling decisions.

Feature Importance Insights:

1. **channel_quality** – The dominant predictor in this dataset. Poor channel conditions directly increase both retransmissions and AoI, consistent with the SINR-dependent success probability in Farag et al. (2023).
2. **packet_loss_probability** – Strongly linked to AoI; lost packets force retransmissions, directly aging the information at the receiver.
3. **transmission_probability** – Higher transmission probability reduces staleness by increasing update frequency on the shared channel.

4. **interference_proxy / num_devices** – Secondary contributors: more devices and higher interference degrade the channel environment.

3d. Predictions for New Hypothetical Configurations

```
# ── Hypothetical network configurations ──────────────────────────────
new_configs = pd.DataFrame({
    'num_devices':              [5,     20,    12],
    'transmission_probability': [0.90,  0.30,  0.60],
    'interference_proxy':       [-0.5,  1.5,   0.0],
    'channel_quality':          [0.95,  0.30,  0.65],
    'traffic_type_encoded':     [0,     1,     0],
    'packet_loss_probability':  [0.05,  0.90,  0.45],
})

descriptions = [
    'Config 1: Low interference, high channel quality, low PLP  (ideal)',
    'Config 2: High interference, poor channel, high PLP         (stressed)',
    'Config 3: Moderate conditions, balanced trade-off           (typical)',
]
```